Capstone Project Synopsis

On

PeerPath – A Campus Peer Skill-Exchange Platform

Submitted during _____ semester in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Information Technology

by

Inderpal Singh (25001011020)

Under supervision of

Dr. Parul Tomar



Department of Computer Science & Engineering

FACULTY OF INFORMATICS & COMPUTING

J.C. Bose University of Science & Technology, YMCA

Faridabad – 121006

JULY-DEC 2026

TABLE OF CONTENTS

Chapter 1: Introduction.....3

1.1 Brief of the Project.....3

1.2 Technologies Used.....3

Chapter 2: Proposed Methodology.....4

2.1 System Architecture.....4

2.2 Data Design.....4

2.3 Core Modules and Workflow.....4

2.4 Development Methodology.....5

Chapter 3: Applications and Scope.....6

3.1 Applications.....6

3.2 Future Scope.....6

CHAPTER 1: INTRODUCTION

1.1 BRIEF OF THE PROJECT

PeerPath is a campus-focused peer skill-exchange web application designed to let college students trade knowledge instead of money. The platform allows a student to list the skills they can teach and the skills they wish to learn, and then matches them with compatible peers on campus. Once matched, students can request a free one-hour session, chat with their peer, meet over a video call, and rate each other after the session concludes, gradually building a verified skill profile.

The project addresses three problems commonly faced by college students in India: the lack of structured peer guidance (learners jumping between disconnected YouTube or Udemy resources with no accountability), high dropout rates when a learner gets stuck with no one to turn to, and the absence of any zero-cost knowledge-exchange mechanism through which a student skilled in one domain (for example, Python programming) can trade that skill for another (for example, video editing) without any monetary transaction.

PeerPath solves this through a skill-barter model. Every student declares two skills they can teach and two skills they want to learn. The discovery feed surfaces peers with complementary skill sets, after which either party can initiate a session request. Confirmed sessions come with an automatically generated video-meeting link, an in-app chat window for coordination and file sharing, and a lightweight admin system to moderate the community. The target audience is college students (B.Tech, BCA, B.Sc, Diploma) aged 18–24 across India, with an initial pilot planned in the Delhi NCR region.

1.2 TECHNOLOGIES USED

Frontend

- HTML5 – page structure and markup
- CSS3 – styling and responsive layout
- Vanilla JavaScript (ES6) – all application logic, DOM manipulation, and event handling, without any external UI framework

Data Storage

- Browser localStorage – stores application data (users, skills, session requests, chat messages) as JSON, entirely on the client side, removing the need for a server-side database

Video Sessions

- Jitsi Meet (meet.jit.si) – free, account-less video conferencing, linked directly from a confirmed session

Hosting

- Static hosting (for example, GitHub Pages or Vercel) – the entire application is a static site of HTML, CSS, and JS files with no backend server to deploy or maintain

CHAPTER 2: PROPOSED METHODOLOGY

2.1 SYSTEM ARCHITECTURE

PeerPath is implemented as a fully client-side, static web application. There is no backend server or external database; all pages are built with plain HTML, CSS, and JavaScript, and all application data is read from and written to the browser's localStorage. This keeps the architecture deliberately simple and appropriate for a college project: the entire application can be opened directly in a browser or hosted on any static file host, with no server setup, API layer, or database administration required.

A small set of JavaScript modules handles distinct responsibilities — one for managing the current logged-in user, one for reading and writing skill profiles, one for handling session requests, and one for rendering the discovery feed and chat UI. Since the app runs entirely in one browser, a simple “switch active user” control is used in place of real multi-device login, so that the two sides of a peer interaction (for example, sending and accepting a session request) can both be demonstrated within a single browser session.

2.2 DATA DESIGN

In place of database tables, data is organised as JSON arrays stored under a small number of localStorage keys. A users key holds each student's profile — name, college, year, branch, the skills they can teach, the skills they want to learn, and their availability.

A sessions key holds an array of session-request objects, each carrying a requester, a target user, the skill being exchanged, a status (pending, confirmed, completed, or declined), the scheduled date and time, and an array of chat messages for that session. This mirrors the structure of the original relational schema closely enough to demonstrate the same data relationships, while avoiding the overhead of setting up and hosting an actual database for a project of this scope.

2.3 CORE MODULES AND WORKFLOW

Profile and Discovery Module

A student fills a simple profile form — name, college details, skills to teach, and skills to learn — which is saved to the users array in localStorage. A discovery page then reads that array and renders a card for every other user, with basic search and filtering by skill, so a student can browse and find a suitable peer.

Session Request Module

From a peer's profile card, a student can send a session request specifying the skill and a preferred date and time. This creates a new entry in the sessions array with status pending. Switching to the target user's view shows the incoming request, which can be accepted (status becomes confirmed) or declined (the entry is removed). Once confirmed, either user can later mark the session as completed.

Chat Module

Each confirmed session includes a lightweight chat window. Messages typed by either user are appended to that session's messages array in localStorage and re-rendered on screen, giving the look and feel of real-time chat within the scope of a single-browser demo.

Video Session Module

When a session is confirmed, a “Join Call” button opens a Jitsi Meet link, giving both peers a free, no-signup video call for their one-hour session without building any custom video infrastructure.

Rating and Admin Module

Once a session is marked completed, each participant can rate the other on a 1–5 scale; the rating is stored against the peer's user record in localStorage and averaged with their previous ratings to keep a running score. A simple admin view — unlocked for a

designated admin account — reads the same users and sessions arrays to show basic platform-wide counts and lets the admin remove a problematic user profile, without needing any separate backend logic.

2.4 DEVELOPMENT METHODOLOGY

The project was built as a single static front-end codebase, developed page by page: profile and discovery first, then session requests, then chat. Each feature was tested directly in the browser using localStorage as the data layer, with no build tooling or server required. This straightforward, file-based workflow keeps the project easy to run, demonstrate, and extend, which is well suited to the scope and timeline of a semester capstone project.

CHAPTER 3: APPLICATIONS AND SCOPE

3.1 APPLICATIONS

- Enables students within a college or campus to exchange skills such as programming, design, music, or public speaking without any monetary cost.
- Provides a structured, accountable alternative to unstructured self-learning from scattered online videos, by pairing learners with an actual peer mentor.
- Reduces learning dropout by giving students a real person to turn to when stuck, rather than an anonymous video or forum.
- Helps students build a verifiable, rated skill profile over time that reflects both what they can teach and what they have learned.
- Offers a low-cost model for colleges or student clubs to encourage peer mentorship and cross-disciplinary learning on campus.

3.2 FUTURE SCOPE

- Extending the platform beyond a single-campus pilot to a multi-college network, allowing skill exchange across institutions.
- Introducing gamification elements such as badges, streaks, and leaderboards to increase sustained engagement.

- Adding skill-verification mechanisms, such as peer endorsements or short assessments, to strengthen the credibility of listed skills.
- Moving from browser localStorage to a proper backend with a real database (for example, Node.js/Express with PostgreSQL or MongoDB), enabling true multi-user, multi-device access instead of a single-browser demo.
- Adding real account-based authentication (with password hashing and, eventually, OAuth login) once a backend is introduced.
- Building a native mobile application to widen accessibility beyond the current web-only interface.
- Integrating an in-house video-calling solution to reduce dependency on third-party services and enable session recording for later review.